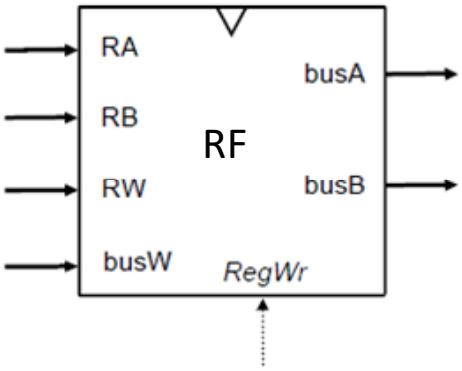
REGISTER FILE FOR A MIPS PROCESSOR

Lab Assignment #1



Computer Architecture and Organization 2
Grado en Ingeniería Informática

Luis M. Ramos Alejandro Valero José Luis Briz Javier Resano
luisma@unizar.esalvabre@unizar.es alvabre@unizar.es briz@unizar.es briz@unizar.es
jresano@unizar.es jresano@unizar.es

Escuela de Ingeniería y Arquitectura Universidad Zaragoza

Departamento de Informática e Ingeniería de Sistemas Universidad Zaragoza

1 SUMMARY

In this assignment you have to design a register bank[1] for a MIPS processor. We will follow a modular design procedure, using combinational blocks: registers, multiplexers and decoders. Once designed, you must test it and perform a timing analysis.

***Take a seat at the lab** and place your pre-lab work (Sec. 2.1.) on the table, so that the instructor can see it. Before starting to work with Logisim, log in to Moodle from one of the computers in the lab. Complete the **internship contract** and **check in at the attendance control of the session**.*

The practice ends when the log bank is working correctly and you have submitted through Moodel the **circuit of section 2.2.j** and the **results of section 3.**

2 DESIGN OF A 32X32 REGISTER BANK

2.1 PRE-LAB: FIRST STEPS WITH LOGISIM AND MODULAR DESIGN

a) Read the resource "Getting Started with Logisim" and watch the video "Logisim Beginner's Tutorial", available in Moodle.

b) Draw up a pen-and-paper version of a bank with four 32-bit registers (**BR4**): use four 32-bit registers, two 4:1x32 MUX multiplexers and one 2:4 DECOD decoder. The register bank must allow two reads and one write per cycle. The **RA** and **RB** input ports indicate which registers are to be read, obtaining the data on **busA** and **busB**. If **RegWr=1** the data in **busW** will be written in the **RW** register on the rising edge at the end of the cycle.

c) Design a bank of 32 32-bit registers (**BR32**) using 8 **BR4**, two 8:1x32 MUX and a 3:8 DECOD. Remember that it must allow two reads and one write in each cycle. Use the same input and output names as in **BR4**.

2.2 LABORATORY WORK: DESIGN WITH LOGISIM

a) Place 4 registers (component folder *Memory*) in a Logisim sheet. Configure them as 32 bit registers (*properties/Data Bits*). Place a DECOD 2:4 (*Plexers/Decoder* folder, *properties/Select Bits→ 2* and *properties/Disabled Output→ Zero*) whose outputs control the write permissions of the registers (**en**). Interconnect the ***reset***, **clock** and **data** input signals.

b) Place two MUX 4:1x32 (*Plexors/Multiplexer, properties/Select Bits→ 2*, *properties/Data Bits→ 32* and *properties/Include Enable? → No*) that select the data from one of the registers.

c) Place inputs for **reset**, **clock** and **busW**, **RA**, **RB**, **RW** and **RegWr** signals**.** Set them to the correct number of bits (*properties/Data Bits*) and name them (*properties/Label*). Connect them where appropriate. Connect outputs to the MUX, set them to 32 bits and name them (**busA** and **busB**).

d) We already have a bank of 4 registers. Check that it works correctly by performing several writes (**RW**, **busW**, **RegWr**), reads (**RA**, **RB**) and deletes (**reset**).

[1]Just take "register file" and "register bank" as the same concept / component for the purposes of this course. As (if) you go further, you'll learn the differences.

e) Rename the circuit as **BR4** (click on main / *properties* / *Circuit Name*). You can change the appearance of the package and the position of the pins by clicking . Click to return to the circuit layout.

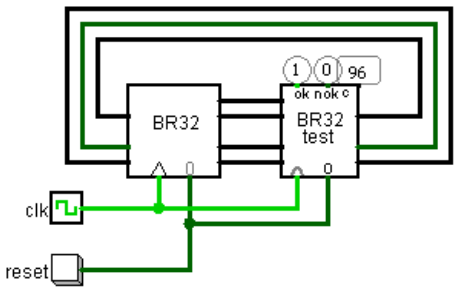
f) Add a new circuit (*Project* / *Add Circuit*). Name it **BR32**. Mark it as the main circuit (*Project* / *Select As Main Circuit*).

g) Place 8 **BR4**. Add a DECOD 3:8 so that the write is performed correctly in one of the 32 registers. Add MUX 8:1x32 so that the two reads are performed correctly.

h) Place inputs and outputs for all signals of the 32-bit register bank (also for the clock signal). Set them to the appropriate number of bits. Use separators to separate the necessary bits (*Wiring/Separator*) and connect them where they belong. Use tunnels (*Wiring/Tunnel*) to connect signals through names.

i) Switch to *simulation* mode and perform several writes and reads in different registers. To view the value of each signal you can add *Wiring/Ver* components or directly click on the signal. You can also double-click on a **BR4** to enter it and check the contents of the registers.

j) Before making the delivery of the circuit you must pass a test vector. To do this:
   a. Make sure that you do not use any *Clock* component (the clock signal must be connected to a normal input).
   b. Add a new circuit. Name it **main**. Mark it as the main circuit.
   c. Go to the Moodle resource "Delivery NIA-BR32test", download the file "BR32test.circ" and save it in the same directory as your .circ file. Add it as an external component (*Project / Load Library / Logisim Library / Select file "BR32test.circ"*).
   d. In the **main** circuit place a **BR32test** component and your **BR32** component. Place a *reset* button and a *Clock* component and connect them to the inputs of both components.
   e. Connect the **BR32test** outputs (**RegWr**, **busW**, **RW**, **RA**, **RB**) to the **BR32** inputs. Connect the **BR32** outputs (**busA**, **busB**) to the **BR32test** inputs.
   f. Place 3 *Ver* components on the **ok**, **nok**, and **c** outputs of **BR32test**. Set the *Ver* component of the **c** signal to "Unsigned decimal".
   g. Activate the clock (Ctrl-K). If after 96 cycles the **ok** signal is activated, your circuit is working correctly. If at some point the **nok** signal is activated and **c** stops your circuit is faulty.



What does the test vector do?

cycle (c)	
0-31	r_i=1 sin RegWr
1-32	(r_i ==0) in RA and RB
33-64	r_i=i+1
34-64	(r_o ==1) in RA and RB
65-96	(r_i ==i+1) in RA and RB

k) When your circuit has passed the test vector you can submit it through the Moodle resource "NIA-BR32test submission"

3 TEMPORAL ANALYSIS

a) Performs a temporal analysis of the combinational blocks (MUX 4:1, MUX 8:1, DECOD 2:4, DECOD 3:8), calculating the delay of the paths between all inputs and outputs of each block. It assumes an implementation following the expression $Z = X_0 * \overline{S_1} * \overline{S_0} + X_1 * \overline{S_1} * S_0 + \cdots$ for the MUX and$Z_0 = en * \overline{S_1} * \overline{S_0}$;$Z_1 = en * \overline{S_1} * S_0$; ... for the DECOD. The delays of the gates used are: d_{NOT}= 5 ps; $d_{OR2\text{-}4}$= 20 ps; $d_{AND2\text{-}4}$= 20 ps; d_{REG}= 50 ps; $tsetup_{REG}$= 30 ps. Note that we have OR gates with a maximum of 4 inputs.

b) Identify the combinational paths involved in reading the **BR32** (RA→ busA and REG→ busA) and calculate their delays ($d_{RA\rightarrow busA}$, d_{busA}). Note that the RA bits have different paths, and you should obtain the maximum delay of all of them.

c) Identify the combinational paths involved in writing the **BR32** (RW→ REG, busW→ REG and RegWr→ REG). How far in advance of the clock edge does the value of each input ($tsetup_{RW}$, $tsetup_{busW}$, $tsetup_{RegWr}$) have to be set?

d) When you have all the delays and *setup* times, enter them through Moodle's "BR32 Temporal Analysis" resource.